The Abstract Robot Object Model

This document explains the design philosophy behind the abstract robot model in this codebase and how concrete robots map onto it.

Reference classes:

- Robot
- DriveTrain
- MechAssembly
- MechComponent
- BillyRobot
- Wildbots2025
- MecanumDrive
- StandardTankDrive
- BillyMA
- CascadeArm

Core Idea

The object model in this project does two jobs at the same time:

1. It defines what every major robot part **must** do.
2. It strongly shapes **how** those responsibilities get implemented and connected.

That distinction matters.

- The abstract classes define required capabilities and required seams.
- The concrete classes supply the hardware choices, control logic, and behavior details.

So the model is not just a taxonomy of classes. It is an architectural rule set.

Philosophy

The codebase is built around a simple premise:

- every layer should have a clear responsibility
- each layer should talk to the layer directly below it
- high-level code should command behaviors, not manipulate raw hardware
- concrete robots should vary by composition, not by rewriting the whole control flow

In practice, that means:

- `OpMode` owns FTC lifecycle
- `Robot` owns whole-robot composition
- `DriveTrain` owns chassis movement
- `MechAssembly` owns subsystem coordination
- `MechComponent` owns individual mechanisms

This is an object model built around **roles**, **contracts**, and **control flow**.

The Layered Structure

The abstract model looks like this:

```
OpMode
  -> Robot
       -> DriveTrain
       -> MechAssembly
            -> MechComponent
            -> MechComponent
            -> MechComponent
       -> robot-wide sensors/helpers
```

And the autonomous-facing model looks like this:

```
Robot.AutonomousRobot
  -> DriveTrain.AutonomousDriving
  -> MechAssembly.AutonomousMechBehaviors
       -> MechComponent.AutonomousComponentBehaviors
```

That second structure is important. The codebase does not treat autonomous as an afterthought. It gives each layer its own autonomous surface.

What the Abstract Classes Require

`DriveTrain`

The abstract `DriveTrain` contract requires every concrete drive train to provide:

- a manual drive entry point: `drive(Gamepad gamepad)`
- a telemetry entry point: `updateTelemetry(Telemetry telemetry)`
- an autonomous-facing API: `getAutonomousDriving()`

That means any drive train, regardless of wheel geometry, must support the same three modes of use:

- TeleOp control

- telemetry reporting
- autonomous command access

What changes between implementations is the actual motion logic.

Examples:

- `MecanumDrive` interprets the gamepad as omnidirectional motion plus turn
- `StandardTankDrive` interprets the gamepad as left and right tread power

The abstract class says, "you must be drivable, observable, and automatable." The concrete class decides what that means physically.

`MechAssembly`

The abstract `MechAssembly` contract requires every concrete assembly to provide:

- a way to receive operator instructions: `giveInstructions(Gamepad gamepad)`
- a way to report telemetry: `updateTelemetry(Telemetry telemetry)`
- a way to expose autonomous subsystem behaviors: `getAutonomousBehaviors()`

This tells us what an assembly is supposed to be:

- not a raw mechanism
- not the whole robot
- a coordinator of multiple mechanisms

The base type does not say how many components exist or what they are. It says that if something is an assembly, it must know how to:

- fan out instructions
- collect status
- expose a higher-level autonomous surface

`MechComponent`

The abstract `MechComponent` contract requires every concrete component to provide:

- a control strategy at construction time
- a manual motion/update entry point: `move(Gamepad gamepad)`
- a telemetry entry point: `update(Telemetry telemetry)`
- an autonomous-facing API: `getAutonomousBehaviors()`

This says that a component is never just a hardware handle.

A component must be:

- controllable
- observable

- autonomous-capable

It also says something about implementation style:

- components are expected to use a strategy object or lambda
- gamepad-to-hardware translation belongs inside that strategy boundary

That is a strong design choice. The base model is pushing concrete implementations toward inversion of control.

`Robot`

The abstract `Robot` contract requires every concrete robot to provide:

- a concrete autonomous wrapper: `getAutonomousRobot()`

It also supplies two concrete whole-robot algorithms:

- `update(gamepad1, gamepad2)`
- `updateTelemetry(telemetry)`

Those methods are more important than they first appear.

`Robot.update(...)` fixes the default runtime orchestration:

1. driver controls go to the drive train
2. mechanism controls go to the mech assembly

`Robot.updateTelemetry(...)` fixes the default telemetry orchestration:

1. drivetrain telemetry is gathered
2. assembly telemetry is gathered
3. telemetry is updated

So `Robot` is not only a contract. It is also a control-flow template.

What the Model Says Must Happen

The abstract types encode a set of non-negotiable expectations.

Every robot must be decomposed into major subsystems

Because `Robot` owns a `DriveTrain` and a `MechAssembly`, the model assumes every concrete robot has at least these two top-level responsibilities:

- mobility
- mechanisms

Even if a particular robot is simple, the architecture still wants those concerns separated.

Every subsystem must support both TeleOp and autonomous use

Every major abstract type includes an autonomous-facing nested type:

- `DriveTrain.AutonomousDriving`
- `MechAssembly.AutonomousMechBehaviors`
- `MechComponent.AutonomousComponentBehaviors`
- `Robot.AutonomousRobot`

That means autonomous support is not optional at the architectural level. The model expects every layer to expose purposeful behaviors for autonomous code.

Manual control and autonomous control should use different APIs

The model deliberately separates:

- `drive(Gamepad)` from autonomous drive verbs
- `move(Gamepad)` from autonomous mechanism verbs

This is a major philosophical point.

TeleOp code answers:

- what is the operator doing right now?

Autonomous code answers:

- what action should happen next?

The object model keeps those worlds separate.

Higher layers should not own lower-level hardware details

Nothing in `Robot` says how many motors a drive train has. Nothing in `MechAssembly` says how a component talks to its motor or servo. Nothing in an autonomous strategy should need to know a motor configuration name.

That is intentional. The model is forcing information hiding.

What the Model Says About How Things Get Done

The abstract model also constrains implementation style.

This is the deeper part of the design.

1. Inheritance defines roles

Inheritance is used here to define categories of things:

- a drive train is a kind of `DriveTrain`

- a mechanism group is a kind of `MechAssembly`
- a hardware wrapper is a kind of `MechComponent`
- a whole robot is a kind of `Robot`

That defines what a class is responsible for.

But inheritance is not doing all the work.

2. Composition creates the real robot

The actual robot is created through composition:

- `BillyRobot` is composed from `MecanumDrive` and `BillyMA`
- `Wildbots2025` is composed from `MecanumDrive` and `CascadeArm`
- `BillyMA` is composed from intake, pusher, flywheel, and turret components
- `CascadeArm` is composed from lift motor, drawbridge motor, and claw components

This is an important principle of the codebase:

Inheritance defines the allowed shapes. Composition builds the real machine.

3. Strategy objects define local control behavior

At the component layer, the model expects a strategy to translate intent into hardware commands.

Examples:

- `SpinnyIntake` defines `SpinnyIntakeControlStrategy`
- `DoublyLimitedMotor` defines `DoublyLimitedMotorControlStrategy`
- `PusherServo` and `Turret` do the same for their own hardware

Then the assembly injects the actual behavior, usually with lambdas.

That means the object model is not only saying, "components must move." It is also saying, "components should get their control policy from outside."

This has real consequences:

- components become reusable
- control mappings stay close to subsystem composition
- different robots can reuse the same component with different behavior policies

4. The base `Robot` class acts like a template method

`Robot.update(...)` is a fixed algorithm:

```
driveTrain.drive(gamepad1);
mechAssembly.giveInstructions(gamepad2);
```

Concrete robots usually do not replace that algorithm. They extend around it.

Billy shows this clearly:

- it calls `super.update(gamepad1, gamepad2)`
- then it layers in robot-wide target tracking

So the base class defines how the main loop is structured, while subclasses optionally add cross-cutting behavior.

That is a template-method style design even though the class name does not say so explicitly.

5. Autonomous behavior is exposed through nested behavior objects

Each layer exposes a nested autonomous type instead of just reusing the TeleOp API.

This changes how autonomous code gets written.

Instead of saying:

- "grab the flywheel motors and set raw power"

the architecture encourages code like:

- `robot.getAutonomousRobot().mechAssembly.autonFlywheel.setPower(0.6)`

That access path is revealing:

- `robot` gives the whole autonomous surface
- `mechAssembly` scopes the request to the mechanism group
- `autonFlywheel` scopes it to one mechanism
- `setPower()` is the approved action

The object model therefore defines not only the nouns in the system, but the verbs available at each layer.

6. Data flows downward, capabilities flow upward

A useful way to read this architecture is:

- instructions flow downward
- specialized capabilities flow upward

Downward flow:

- OpMode gives input to `Robot`

- `Robot` dispatches to drive and mech layers
- `MechAssembly` dispatches to components
- components command hardware

Upward flow:

- components expose autonomous behavior objects
- assemblies gather those into assembly-level autonomous objects
- robots gather those into robot-level autonomous objects

That is why the autonomous wrappers matter so much. They gather low-level capability into higher-level command surfaces.

Abstract Contracts vs Concrete Choices

The table below shows how the model separates "must do" from "how it is done."

`| Abstract type | What it requires | Concrete implementation decides |`

`|---|---|---|`

`| DriveTain | drive manually, report telemetry, expose autonomous drive behavior | wheel layout, motor mapping, kinematics, IMU use |`

`| MechAssembly | receive mech instructions, report telemetry, expose autonomous assembly behavior | which components exist, how conflicts are resolved, what assembly-level sequences exist |`

`| MechComponent | move, report telemetry, expose autonomous component behavior, accept a strategy | which hardware it owns, how controls map to hardware, what safeguards exist |`

`| Robot | expose a concrete autonomous robot and own major subsystems | which drive train, which assembly, which robot-wide sensors/helpers, what cross-cutting updates run |`

This is the central design philosophy:

- abstract classes specify required responsibilities
- concrete classes implement those responsibilities with real hardware and real policies

How Concrete Implementations Map Onto the Model

Robot level

Concrete mappings:

- `BillyRobot -> Robot`
- `Wildbots2025 -> Robot`

Shared obligations:

- own a drive train
- own a mech assembly
- expose a concrete autonomous robot wrapper

Different choices:

- Billy adds Limelight, IMU setup, and turret targeting
- Wildbots2025 is simpler and only wires drive plus arm assembly

So both satisfy the same abstract role, but at different levels of complexity.

DriveTrain level

Concrete mappings:

- `MecanumDrive -> DriveTrain`
- `StandardTankDrive -> DriveTrain`

Shared obligations:

- accept gamepad input
- report telemetry
- provide autonomous drive behaviors

Different choices:

- mecanum computes omnidirectional wheel powers
- tank directly maps left and right sticks to tread power

The abstract model allows major physical variation without changing the rest of the system.

MechAssembly level

Concrete mappings:

- `BillyMA -> MechAssembly`
- `CascadeArm -> MechAssembly`

Shared obligations:

- fan out gamepad instructions
- gather telemetry
- expose grouped autonomous behaviors

Different choices:

- Billy coordinates shooter-related mechanisms and includes rapid fire sequencing
- CascadeArm coordinates arm motion, drawbridge motion, and claw control

Again, the assembly contract stays constant while subsystem content changes.

MechComponent level

Concrete mappings:

- `SpinnyIntake -> MechComponent`
- `DoublyLimitedMotor -> MechComponent`
- `Turret -> MechComponent`
- `PusherServo -> MechComponent`

Shared obligations:

- own hardware
- accept or use a control strategy
- implement manual update behavior
- expose autonomous verbs
- provide telemetry

Different choices:

- `SpinnyIntake` is a simple motor wrapper
- `DoublyLimitedMotor` adds local safety logic through limit-switch enforcement
- `Turret` and `PusherServo` express servo-specific control surfaces

This is where concrete classes show the most implementation diversity, while still obeying the same conceptual contract.

Why the Nested Autonomous Types Matter

The nested autonomous types can look unusual at first, but they are doing important architectural work.

They create:

- a narrower API for autonomous use
- a cleaner separation between manual and autonomous control
- a typed path from robot to subsystem to mechanism

For example:

- `BillyRobot.AutonomousMecanumRobot`
- `BillyMA.AutonomousBillyMA`
- `SpinnyIntake.AutonomousIntakeBehaviors`

Those types say:

- autonomous callers are allowed to use these behaviors
- autonomous callers should not need raw component internals
- autonomous behavior is part of the object model, not an afterthought utility

The Model Is Both Descriptive and Prescriptive

It is descriptive because it names the major concepts in the system:

- robot
- drive train
- assembly
- component

It is prescriptive because it tells you how to build them:

- compose rather than flatten everything into one class
- separate manual and autonomous APIs
- inject control strategies instead of hard-wiring every policy
- collect lower-level autonomous behaviors into higher-level wrappers
- keep OpModes thin and push behavior into the object model

That is what the user request means by the model defining both **what** things do and **how** things get done.

Practical Reading Rules for Students

When looking at a class in this codebase, ask these questions:

If it extends `Robot`

- Which drive train does it compose?
- Which mech assembly does it compose?
- What robot-wide sensors or helpers live here?
- Does it extend the default update loop or leave it alone?

If it extends `DriveTrain`

- How does it interpret driver input?
- What telemetry does it expose?
- What autonomous drive verbs does it offer?

If it extends `MechAssembly`

- Which components does it coordinate?
- How are operator controls distributed?
- What mechanism conflicts are handled here?
- What assembly-level autonomous behaviors exist?

If it extends `MechComponent`

- What hardware does it own?
- What strategy interface does it require?
- What autonomous verbs does it expose?
- What local safety or telemetry logic belongs here?

Final Interpretation

The abstract robot model is best understood as a layered contract system.

It says:

- every major part of the robot has a defined role
- every role has required entry points
- autonomous behavior must exist at every major layer
- composition is the normal way to create a robot
- strategies are the normal way to customize local behavior
- high-level code should operate on meaningful behaviors, not raw hardware

Concrete classes like `BillyRobot`, `MecanumDrive`, `BillyMA`, and `SpinnyIntake` do not replace that model. They fill it in.

They answer:

- which hardware is used
- which behaviors are exposed
- which policies are chosen
- which sensors and helpers are added

The abstract model provides the shape. The concrete classes provide the machine.